

进程与进程调度

操作系统里，进程是程序的一次执行过程，也是系统分配资源的基本单位。一个程序被加载进内存、分配好 CPU 和内存之后就开始运行，这个正在运行的程序实体就叫进程。CPU 是计算机里最宝贵的资源之一，同一时刻一个 CPU 核心只能运行一个进程，而系统里往往有几十上百个进程都等着用 CPU。进程调度要解决的就是这个问题：按什么规则、在什么时候决定把 CPU 交给哪个进程使用。

进程在生命周期里会在几种状态之间来回切换，最基本的有三种：就绪、运行和阻塞。就绪状态是指进程已经准备好运行、只差 CPU；运行状态是指进程正占用着 CPU 在执行；阻塞状态是指进程在等待某个事件完成（比如等待磁盘读取、等待用户输入），这时候就算给它 CPU 它也没法继续往下走。调度器决定的是从就绪队列里挑哪个进程来运行，而阻塞和唤醒则由进程自己请求的事件决定。

调度目标与评价指标

不同的系统对调度有不同的期待，这些期待往往是互相冲突的。批处理系统关心吞吐量，希望单位时间内完成的作业尽可能多；交互式系统关心响应时间，希望用户敲下命令之后屏幕上尽快有反应；实时系统关心截止时间，希望关键任务一定能在规定时间内完成。除此之外还要考虑公平性，不能让某个进程长期得不到 CPU 而饿死。实际使用的调度算法都是在这些目标之间做折中，没有一个算法能同时把所有指标都做到最优。

先来先服务（F C F S）是最简单的调度算法：就绪队列按到达顺序排队，谁先到谁先运行，运行完或者主动放弃 CPU 之后，队首的下一个进程再上。它公平、容易实现、不会饿死，但有一个明显的缺点：如果队首是一个运行时间很长的进程，后面所有进程都得干等着，平均等待时间会被拉得很长。这种现象叫做护航效应。所以先来先服务适合批处理系统，不适合交互式系统，用户等不起。

短作业优先与时间片轮转

短作业优先（S J F）每次从就绪队列里挑预计运行时间最短的进程来运行，可以证明它能让平均等待时间最短。它的问题有两个：一是预计运行时间很难准确知道，实际系统里只能用历史数据去估计；二是长作业可能被源源不断的短作业插队，一直排不上，也就是饿死。还有一种抢占版本叫最短剩余时间优先，每当有新进程到达就比较剩余时间，如果新来的更短就立刻抢占 C P U。

时间片轮转（R R）是交互式系统里最常用的算法。就绪队列还是按到达顺序排，但每个进程一次只运行一个固定长度的时间片，比如 1 0 毫秒；时间片用完就强制切换到队尾重新排队。这样每个进程都能在有限时间内得到 C P U，响应时间有保证。时间片的长短很关键：太短会导致切换过于频繁，切换本身也要花时间，C P U 真正用于计算的比例下降；太长则退化成先来先服务，交互体验变差。一般取 1 0 到 1 0 0 毫秒之间。

优先级与多级反馈队列

优先级调度给每个进程分配一个优先级，每次挑优先级最高的进程运行。优先级可以是静态的（创建时定好就不再变），也可以是动态的（根据等待时间、资源占用情况不断调整）。它最严重的问题是饿死：如果高优先级进程源源不断，低优先级进程可能永远得不到运行机会。常见的解决办法是老化，也就是让等待越久的进程优先级慢慢升高，这样它最终一定能被调度到。

多级反馈队列把就绪队列分成好几级，第一级时间片最短、优先级最高，越往下时间片越长、优先级越低。新进程一律先进入第一级；如果它在一个时间片内没运行完，就被降到下一级；如果它是因为等待输入输出而主动放弃 CPU，就留在原来的级别。这套机制的效果是：短作业和交互式进程能在高优先级、短时间片的那几级里很快完成，而长计算型进程会逐渐沉到下面，用长时间片慢慢跑完。它不需要预先知道运行时间，是通用系统里最常用的方案。

上下文切换与负载影响

上下文切换是调度的实际动作：把当前进程的 CPU 状态（程序计数器、寄存器、栈指针等）保存到它的进程控制块里，再把要运行的那个进程之前保存的状态恢复进 CPU 寄存器，然后跳转到它上次停下的地方继续执行。整个过程需要内核介入，而且要刷新缓存和地址映射，所以是有开销的，通常几微秒。这个开销不产生任何有用的计算，纯属额外成本，所以调度算法不能过于频繁地切换。

实际系统里调度的效果还取决于负载情况。当就绪队列里进程很少时，几乎所有算法表现都差不多；当进程数量远远超过 CPU 核心数时，算法之间的差异才会明显暴露出来。另外，输入输出密集型进程和计算密集型进程的混合比例也很重要：输入输出密集型进程经常主动放弃 CPU，正好给计算密集型进程腾出计算时间，两者混在一起反而能提高整体利用率。所以现代调度器在挑选进程时，会倾向优先照顾那些刚做完输入输出、现在可以计算一会儿的进程。